PRACTICAL 7

Aim

To implement performance optimization techniques using Caching and Partitioning in Apache Spark with PySpark.

The practical focuses on loading the MovieLens Ratings dataset into a Spark DataFrame, examining its schema and data, calculating average movie ratings, measuring execution time before and after caching, applying cache() and persist(StorageLevel.DISK_ONLY), comparing different partition counts, analyzing repartition() and coalesce(), and observing the effect of these techniques on Spark performance.

Problem Definition

A movie recommendation company processes millions of user ratings to generate personalized movie recommendations. The recommendation system repeatedly processes movie rating data to calculate statistics such as average movie ratings and rating counts.

As the volume of user interaction data increases, repeated processing of the same dataset results in increased execution time and computational overhead. Therefore, Apache Spark caching and partitioning techniques are used to improve application performance.

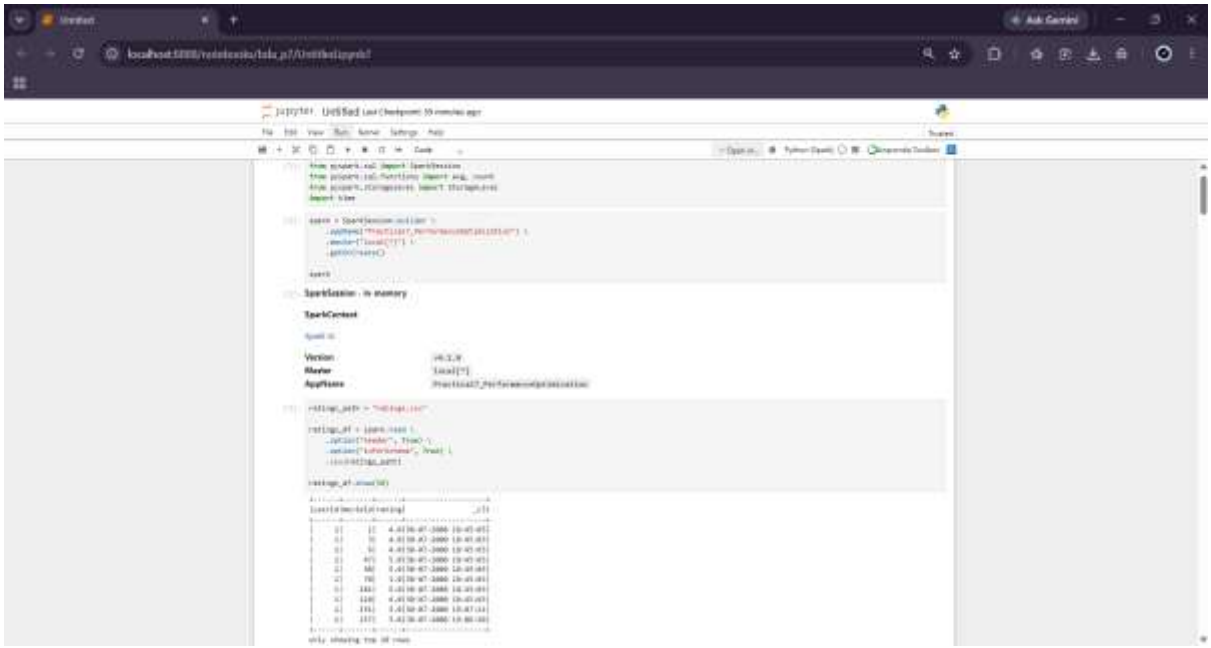
The task is to load the MovieLens Ratings dataset into a Spark DataFrame and perform performance optimization using caching and partitioning. The practical includes creating a SparkSession, displaying the dataset and schema, calculating average movie ratings, measuring execution time before caching, applying cache(), triggering caching using count(), measuring execution time after caching, comparing cache() with persist(StorageLevel.DISK_ONLY), repartitioning the dataset into 2, 4 and 8 partitions, comparing coalesce() and repartition(), and observing scalability using a larger ratings dataset.

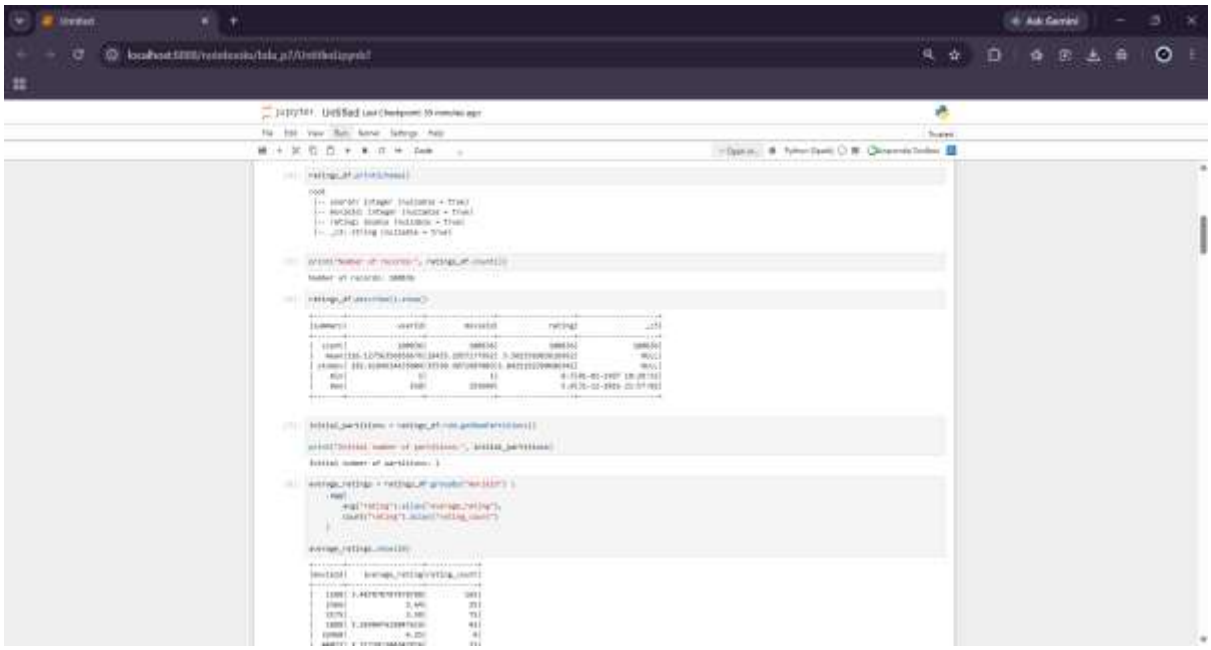
Objectives

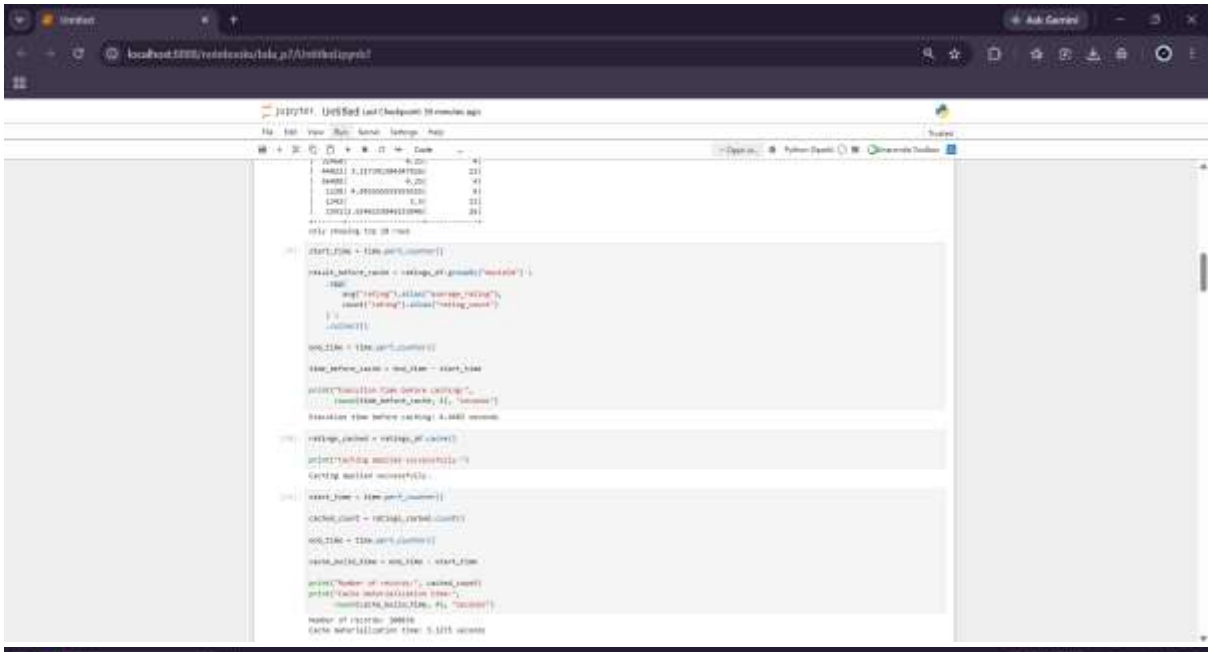
- Understand Apache Spark performance optimization.
- Create and configure a SparkSession.
- Load the MovieLens Ratings dataset into a Spark DataFrame.
- Display and examine the dataset.
- Display the schema of the DataFrame.
- Check the initial number of partitions.
- Calculate average movie ratings.
- Measure execution time before caching.
- Apply cache() to the DataFrame.
- Trigger caching using count().

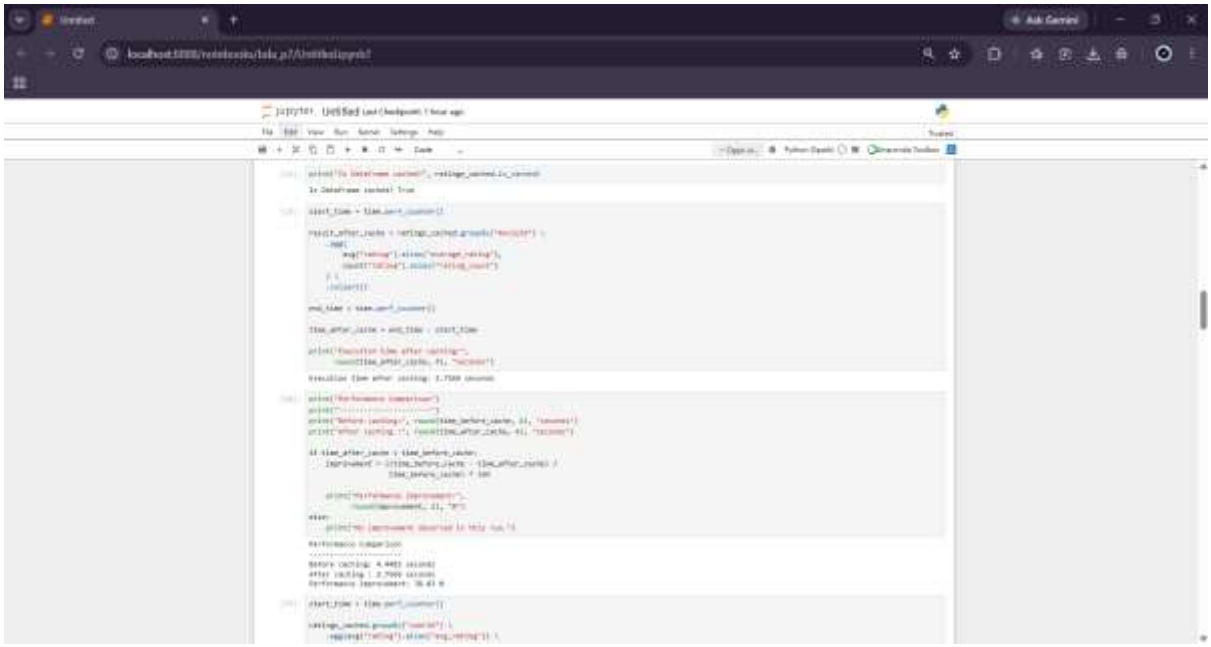
- Measure execution time after caching.
- Compare performance before and after caching.
- Use persist(StorageLevel.DISK_ONLY).
- Compare cache() and DISK_ONLY persistence.
- Repartition the dataset into 2, 4 and 8 partitions.
- Compare execution times for different partition counts.
- Compare coalesce() and repartition().
- Create a larger ratings dataset.
- Observe scalability and performance.
- Understand Spark lazy evaluation.
- Analyze how caching and partitioning affect Spark performance.

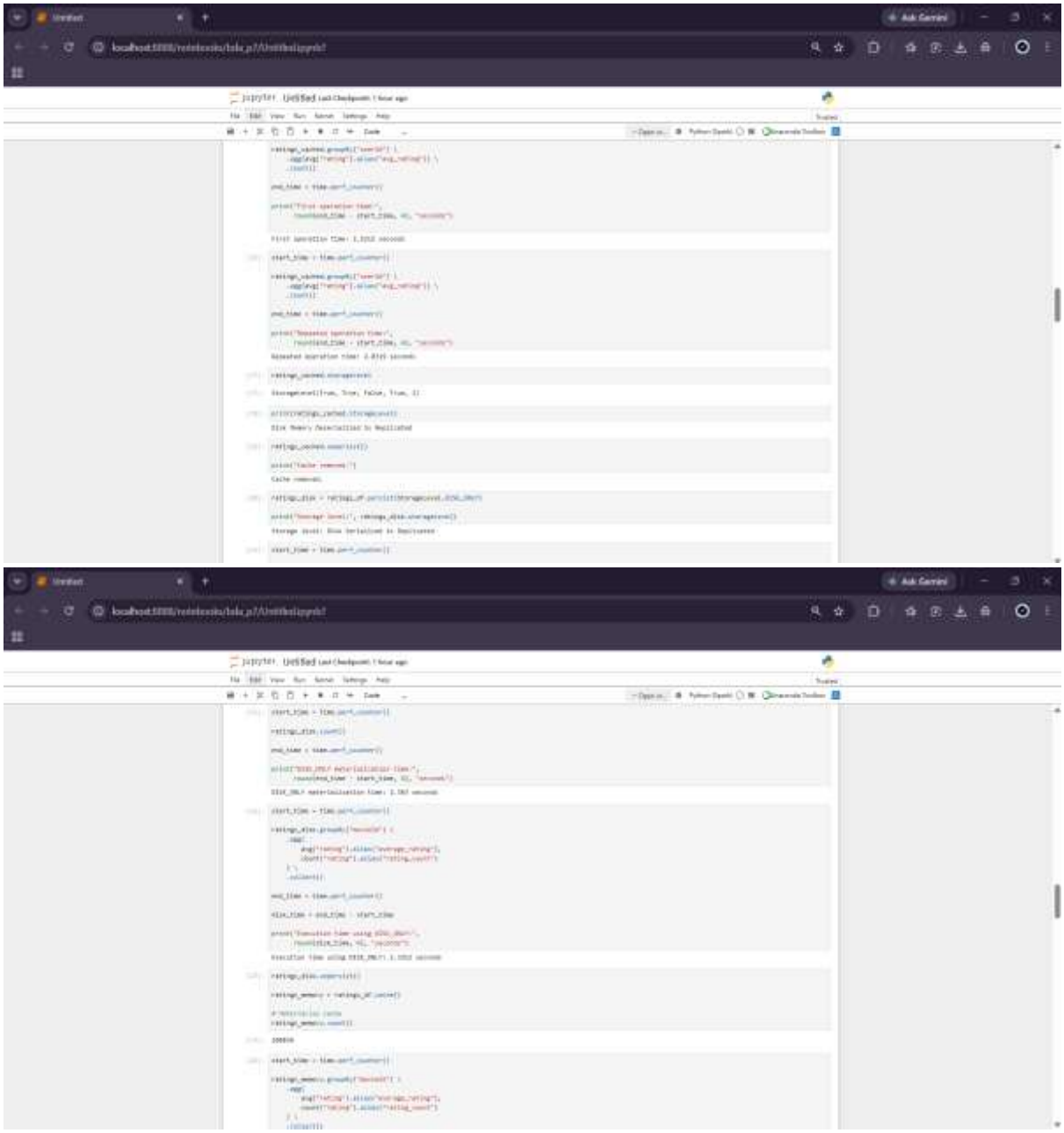
Output

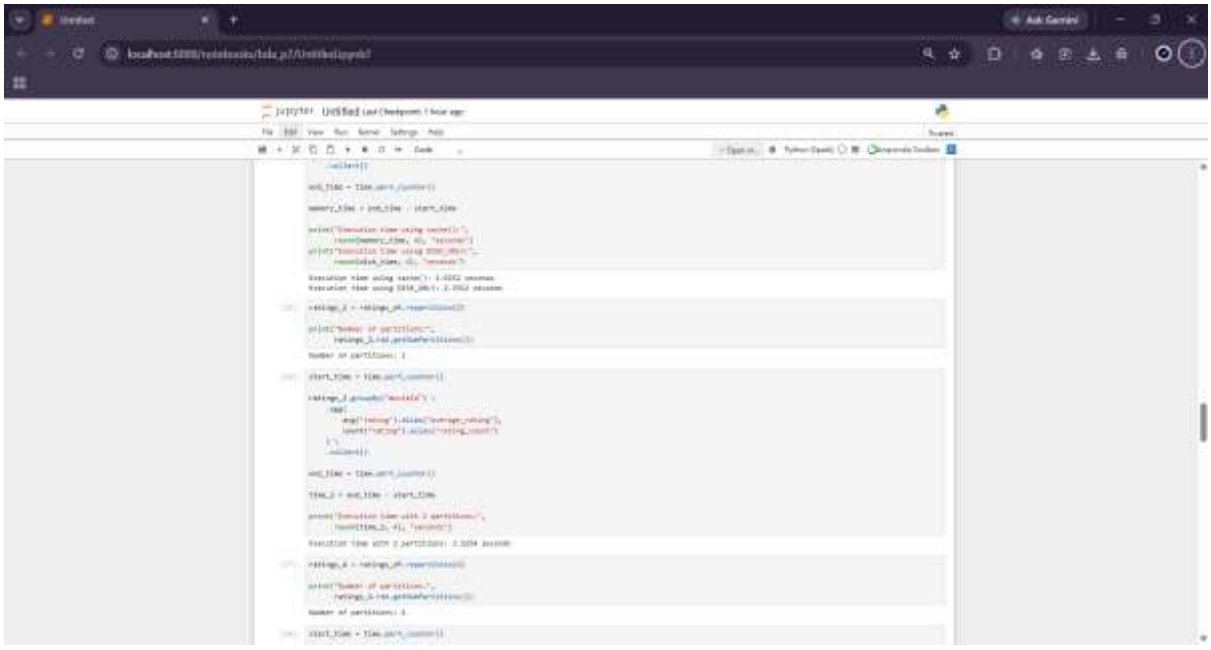


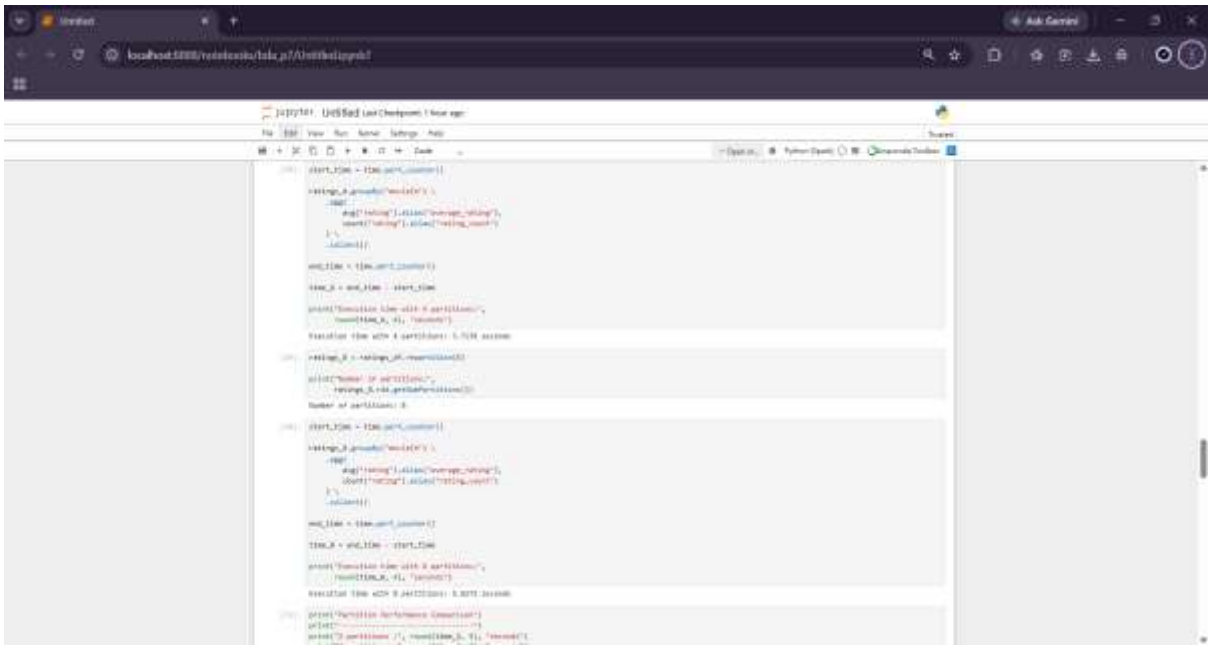


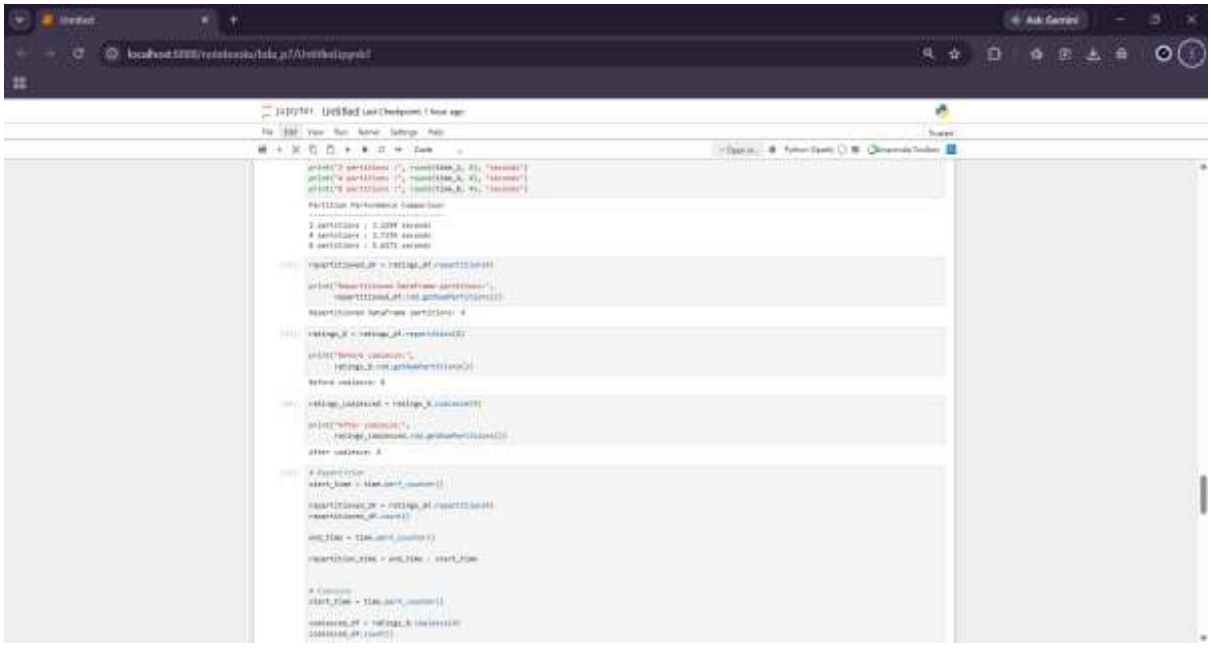


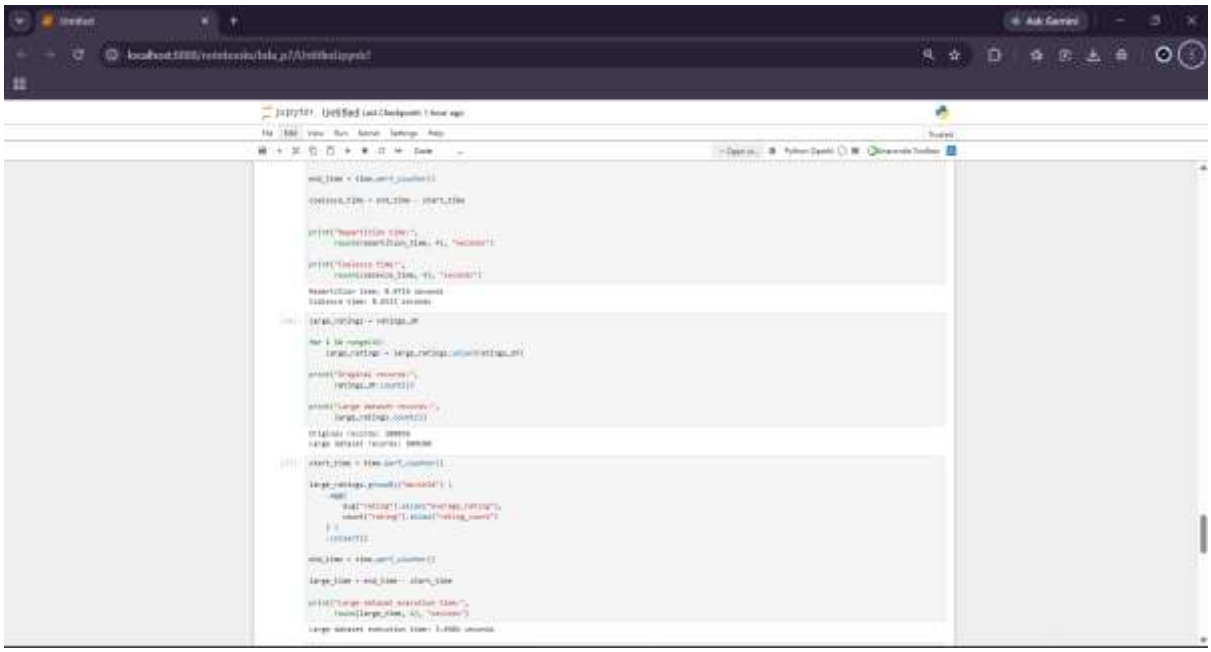


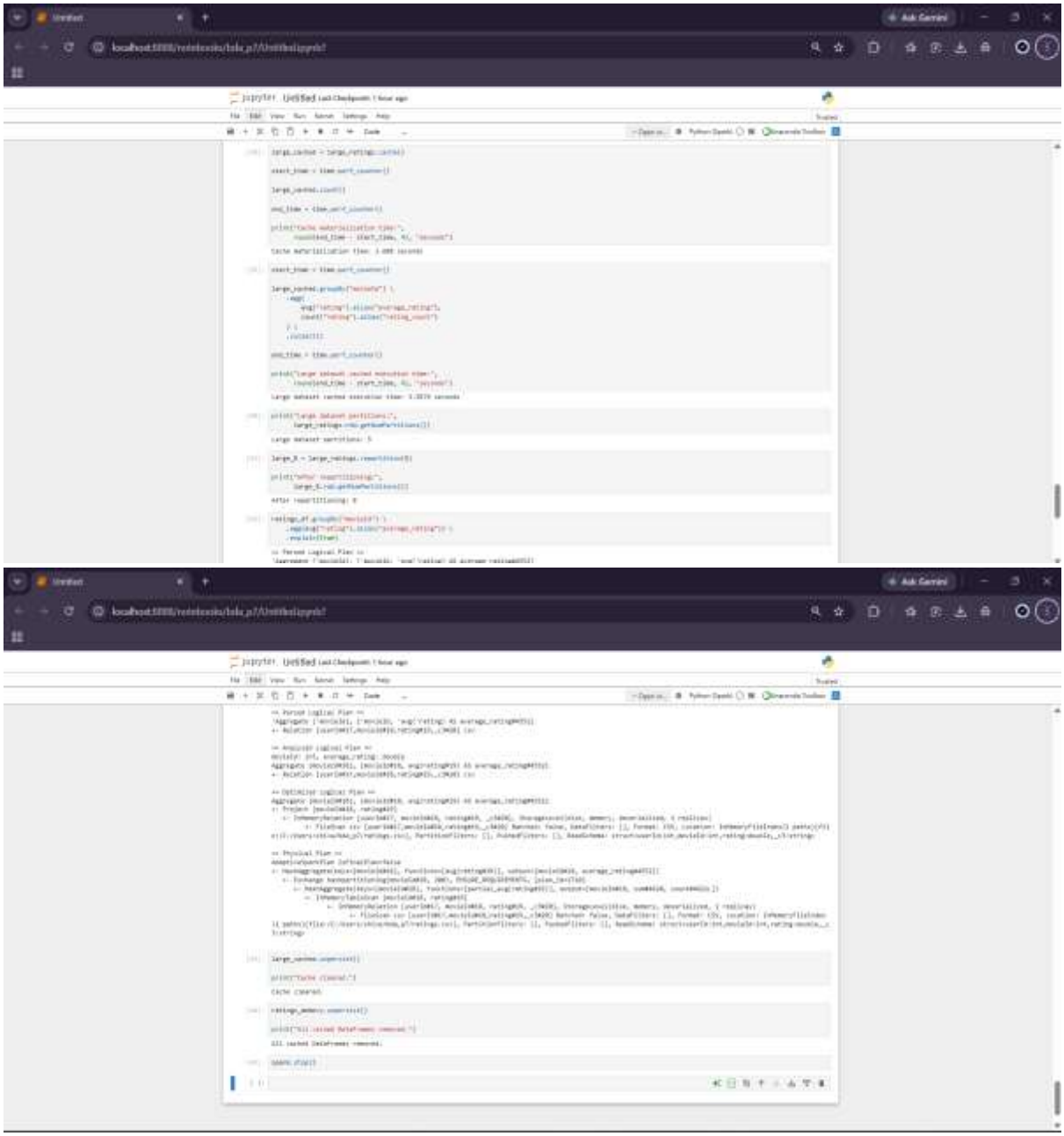












Key Questions / Analysis

Q1. What is caching in Apache Spark and why does it improve performance?

Caching is a performance optimization technique in Apache Spark that stores a computed DataFrame or RDD so that it can be reused for subsequent operations.

When the same dataset is used repeatedly, Spark can access the cached data instead of recomputing the dataset from the original source.

Caching can therefore:

1. Reduce repeated computation.

2. Reduce data processing time.
3. Improve performance for repeated operations.
4. Reduce the need to recompute transformations.
5. Improve interactive analytics and iterative processing.

In this practical, cache() is used to store the MovieLens ratings DataFrame for repeated analysis.

Q2. What is the difference between cache() and persist()?

cache()

The cache() method stores a DataFrame or RDD using Spark's default caching storage level.

Example:

```
ratings_cached = ratings_df.cache()
```

persist()

The persist() method allows the programmer to specify the storage level.

Example:

```
from pyspark.storagelevel import StorageLevel

ratings_disk = ratings_df.persist(StorageLevel.DISK_ONLY)
```

Therefore:

cache()	**persist()**
Uses Spark's default storage level	Allows storage level to be specified
Simple to use	Provides more control
Suitable for frequently reused data	Useful when a specific storage strategy is required

Q3. How does Spark partition data internally?

Spark divides a dataset into multiple partitions. Each partition represents a portion of the complete dataset.

Spark can process different partitions in parallel using different tasks and available CPU resources.

For example:

Dataset

|

+---- Partition 1

|

+---- Partition 2

|

+---- Partition 3

|

+---- Partition 4

The number of partitions can affect the amount of parallelism and the execution performance of a Spark application.

Q4. Why does repartition() improve scalability?

The repartition() operation redistributes data into the specified number of partitions.

For example:

ratings_4 = ratings_df.repartition(4)

This creates four partitions.

With appropriate partitioning, Spark can distribute processing tasks across multiple CPU cores. This can improve parallel processing for larger datasets.

However, excessive partitioning can also introduce additional task scheduling and shuffle overhead.

Q5. How does lazy evaluation affect Spark performance?

Spark uses **lazy evaluation**, which means transformations are not executed immediately.

For example:

df2 = df.filter(df.rating > 3)

The filter() operation does not immediately process all records.

Spark waits until an action such as:

df2.count()

or:

df2.collect()

is executed.

Lazy evaluation allows Spark to construct and optimize an execution plan before actually performing the computation.

Supplementary Problems

1. Compare the performance of cache() and persist(StorageLevel.DISK_ONLY).
2. Repartition the dataset into 2, 4 and 8 partitions and compare execution time.
3. Compare coalesce() and repartition().
4. Create a larger ratings dataset and observe scalability.
5. Analyze Spark's execution plan using explain().
6. Compare execution time before and after caching.

Conclusion

The Performance Optimization Using Caching and Partitioning practical was successfully completed using PySpark and the MovieLens Ratings dataset. The dataset was loaded into a Spark DataFrame and its schema and partitions were examined. Average movie ratings were calculated and execution time was measured before and after applying caching.

The cache() method was used to store frequently accessed data, while persist(StorageLevel.DISK_ONLY) was used to study disk-based persistence. Different partition counts, including 2, 4 and 8 partitions, were tested to analyze their effect on execution time. The practical also compared coalesce() and repartition() and examined the performance of a larger ratings dataset.

The experiment demonstrated the importance of caching, persistence, partitioning, lazy evaluation and parallel processing in optimizing Apache Spark applications. These techniques help improve the efficiency and scalability of data processing workloads, particularly when datasets are large or repeatedly accessed.